Checkpoint quizzes

Five questions after each phase; pass at 80% without notes before the next phase starts. The final exam is at the bottom of the Build spec tab. For a live session, ask Claude "quiz me on StrataKV P2" and it will ask follow-ups like an interviewer.

P0 — Scaffold and CI

1. What's the difference between a header file and a source file?
2. What does each of the four build presets catch or produce?
3. What is undefined behavior, and why run tests under sanitizers?
4. What does scikit-build-core do during `pip install .`?
5. What is CRC32C used for, and what is it *not*?

Key: (1) Headers declare interfaces; source files implement them. (2) `debug`: symbols for debugging; `release`: optimized build for benchmarks; `asan`: memory errors and undefined behavior; `tsan`: data races. (3) Mistakes like use-after-free that C++ doesn't report and that cause unpredictable behavior; sanitizers detect them at runtime. (4) Runs CMake to compile the C++ extension and places it inside the Python package. (5) Detecting accidental corruption and torn writes; it is not secure against deliberate tampering.

P1 — WAL, memtable, recovery

1. Trace a `Put` from call to acknowledgment.
2. Why is truncating a torn WAL tail safe?
3. How do you tell a torn tail from mid-file corruption?
4. Why does the internal key sort sequence numbers descending?
5. What does the release store in the skip list guarantee?

Key: (1) Assign sequence number, encode the record, append to the WAL, `fsync`, insert into the memtable, return OK. (2) The partial record's `fsync` never returned, so it was never acknowledged. (3) A torn tail has no valid records after it; corruption followed by valid records must be reported, not truncated. (4) So the newest version of a key is found first. (5) Any reader that sees the new node pointer also sees the node's fully written contents.

P2 — SSTables and flush

1. What are the four parts of an SSTable, and what is each for?
2. What does a bloom filter answer, and with what error?
3. Why is the old WAL deleted only after the flush is recorded durably?
4. Walk through the read path for a key that exists only in an L0 file.
5. Why can L0 files overlap?

Key: (1) Data blocks (sorted entries), filter block (skip files without the key), index block (find the right data block), footer (locate index and filter, verify with checksum). (2) "Definitely not here" or "maybe here"; false positives about 1%, never false negatives. (3) Until then the WAL is the only durable copy; deleting it first risks losing data in a crash. (4) Miss in active memtable, miss in immutable memtable, then each L0 file newest first: bloom filter, index, data block, found. (5) Each is a separate memtable flush covering whatever keys were written in that period.

P3 — MANIFEST and compaction

1. What does the MANIFEST record, and why is it a log instead of a single file?
2. Walk through the four steps of atomically replacing `CURRENT`.
3. Why can't a tombstone be dropped above the bottom level?
4. How does compaction decide which entries to keep?
5. How do readers avoid having files deleted underneath them?

Key: (1) File additions and removals per level, next file number, last sequence; appending edits is cheap and crash-safe compared with rewriting. (2) Write a temp file, `fsync` it, `rename` over `CURRENT`, `fsync` the directory. (3) An older PUT for that key may exist below and would reappear. (4) A k-way merge in internal-key order keeps the first (newest) entry per user key and drops the rest, plus tombstones at the bottom level. (5) Each read holds a `shared_ptr` to its Version; files are deleted only when no Version references them.

P4 — Concurrency and group commit

1. Explain group commit and why it raises throughput.
2. Why does the leader release the mutex during the WAL write?
3. What is a data race, and how does TSan find one?
4. Why use `shared_ptr` snapshots instead of holding a lock during reads?

5. What would happen if two threads inserted into the skip list at once?

Key: (1) Waiting writers' batches are merged by one leader into one record and one `fsync`; `fsync` cost barely grows with size, so it's amortized. (2) So new writers can queue up during the slow I/O and form the next group. (3) Unsynchronized concurrent access with at least one write; TSan instruments memory accesses and reports conflicting ones without a happens-before relationship. (4) Holding a lock across disk reads would block all writers. (5) Corrupted links; the design allows only one writer at a time, which group commit's single leader guarantees.

P5 — Python package and testing

1. Why release the GIL in `get`, and what must happen first?
2. How are `Status` errors turned into Python exceptions?
3. What does a Hypothesis stateful test do, and what does shrinking add?
4. Why does the fuzzer write each key only once per iteration?
5. Why does the child print a key only after `put` returns?

Key: (1) So other Python threads run during disk I/O; the key must be copied out of the Python object while the GIL is held. (2) The binding checks each `Status` and throws a C++ exception that pybind11 maps to a registered Python exception. (3) Runs random operation sequences against StrataKV and a dict model and asserts they agree; shrinking reduces a failure to the smallest sequence. (4) An unacknowledged later write of the same key could legitimately be on disk and cause a false failure. (5) The printed set must be exactly the acknowledged writes, the only ones guaranteed to survive.

P6 — Benchmarks and README

1. Why compare against SQLite with matched sync settings?
2. What is write amplification, and how do you measure it?
3. Which benchmark shows the bloom filter's value, and why?
4. Why record the command and hardware with every number?
5. Name two limitations of StrataKV you'd state in the README.

Key: (1) Comparing a sync-every-write engine against one that doesn't sync is meaningless; matching makes it fair. (2) Bytes written to disk divided by bytes the user wrote; count bytes written by WAL, flush, and compaction. (3) Random reads of missing keys; filters skip most file reads, which shows clearly against a run with filters off. (4) So

anyone, including an interviewer, can reproduce and trust it. (5) Two levels only, so very large datasets would compact inefficiently; no range scans unless the stretch goal is done; single process only.